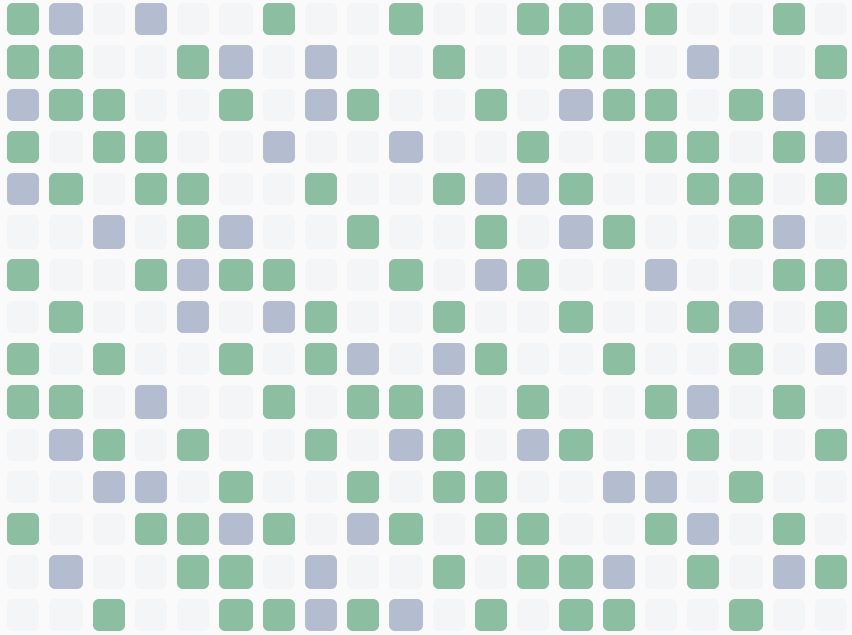
The Memory-Bandwidth Wall

A field guide to min-maxing LLM serving with vLLM

Your dashboard says busy. Your users say slow.

tok/s × bytes = bandwidth



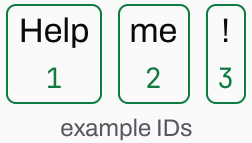
Marcel Wysocki · Senior Principal Architect · Red Hat Global Services
Speaking for myself, not for Red Hat.

CONTAINERDAYS & AI CONTEXT SINGAPORE · 28 OCT 2026

Five stages of a request.

Tokenize

Prompt to token IDs (numbers).



example IDs

Prefill

Prompt tokens in parallel. Compute-heavy.


parallel

Write KV

Keys and values stored per token, per attention layer.



Decode

One token per step. Rereads weights and KV.



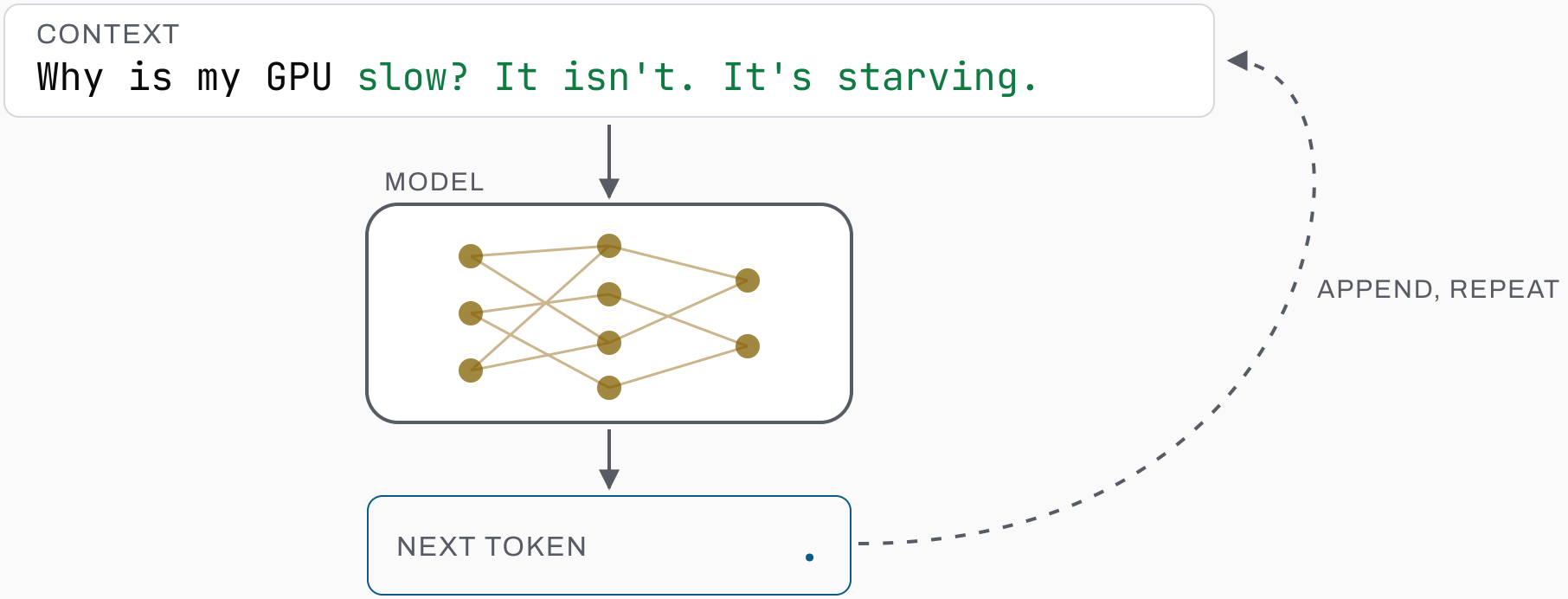
Stream

Tokens back to the user.

Here you go.

Today's session focuses on the **three middle stages**.

Every new token goes around the loop again.



WEIGHTS · READ EVERY STEP

Qwen3.8-27B · bf16 (16-bit) · 55.6 GB checkpoint

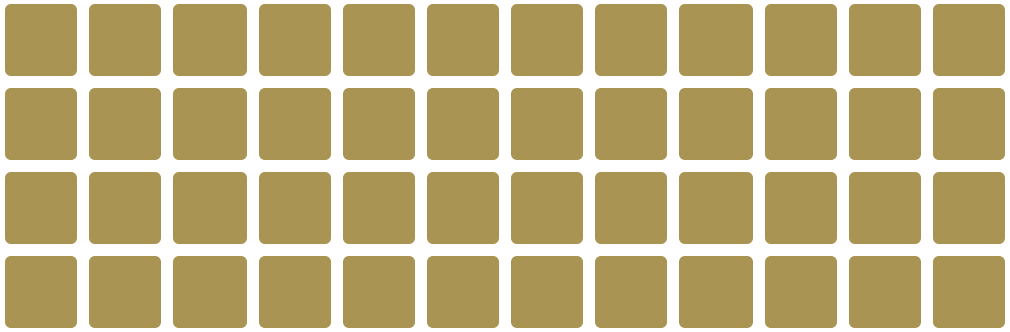
KV CACHE · GROWS EVERY STEP

12 tokens

Schematic. Qwen3.8-27B bf16 checkpoint: 55.6 GB; a decode step reads about 51 GB of it.

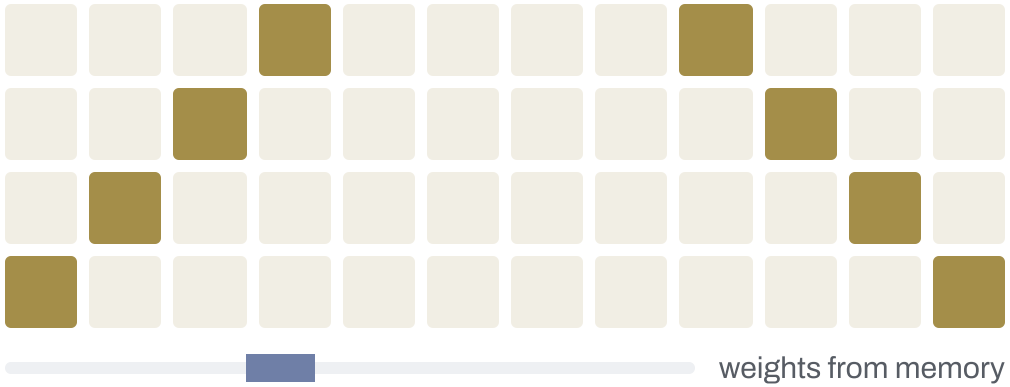
Long-prompt prefill is usually compute-bound. Small-batch decode is usually memory-bound.

Prefill



n tokens in one pass. Matrix units (tensor cores) saturated. Cost scales with **FLOPs** (math operations).

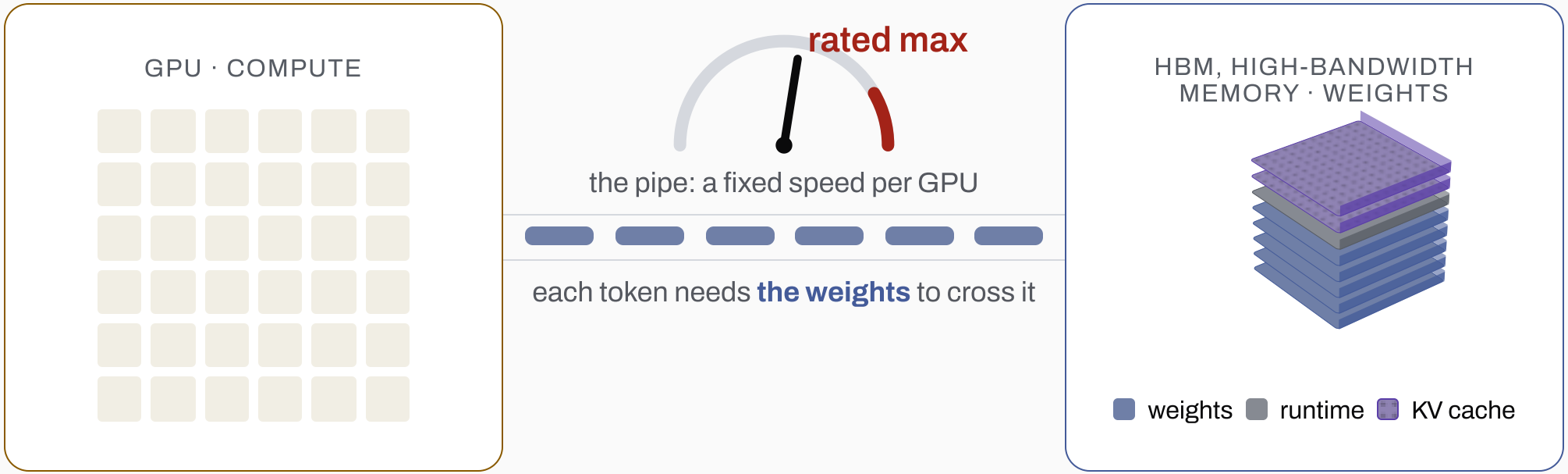
Decode



1 token per pass. Cores wait on memory. Cost scales with **bytes**.

Schematic.

Your GPU isn't slow. It's starving.



Tokens per second is pipe speed divided by bytes per token.

tok/s × bytes = bandwidth

In vLLM: **measure the token gap**, then compare it with the byte budget.

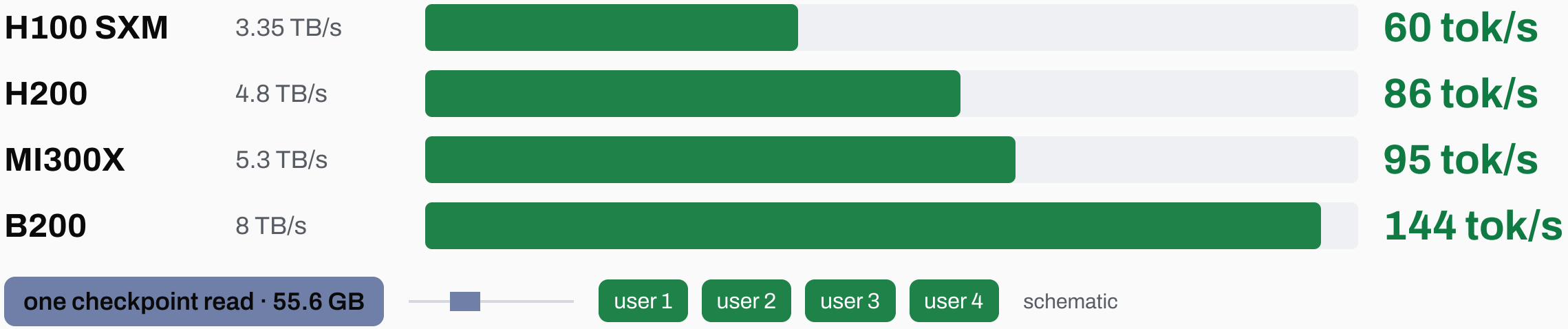
Schematic: one dense-model stream, short context, speculation off. The checkpoint is a first estimate of weight bytes read.

Divide before you tune.

Checkpoint-based estimate: how many times per second the pipe can deliver the stored file.

55.6 GB checkpoint H100 SXM pipe · 3.35 TB/s **GPU** **60 tok/s**

`pipe speed ÷ 55.6 GB checkpoint (Qwen3.8-27B, bf16)`

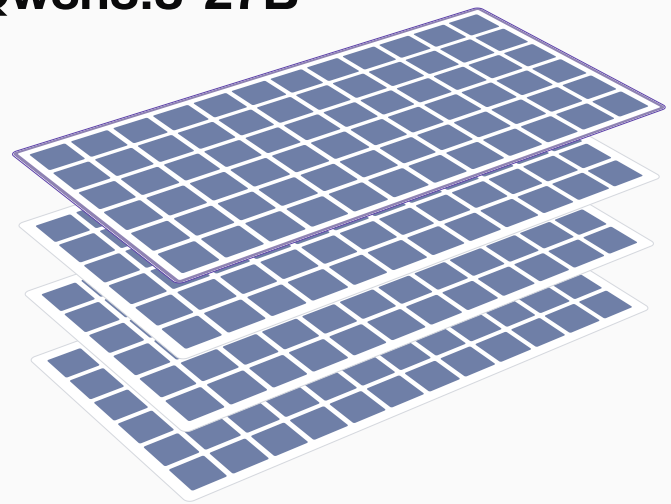


Batching: one read, more users get a token. It never shrinks the read.

Vendor HBM spec sheets ÷ the 55.6 GB checkpoint. Share row schematic.

Dense layers read all their weights.
Expert layers read the ones they pick.

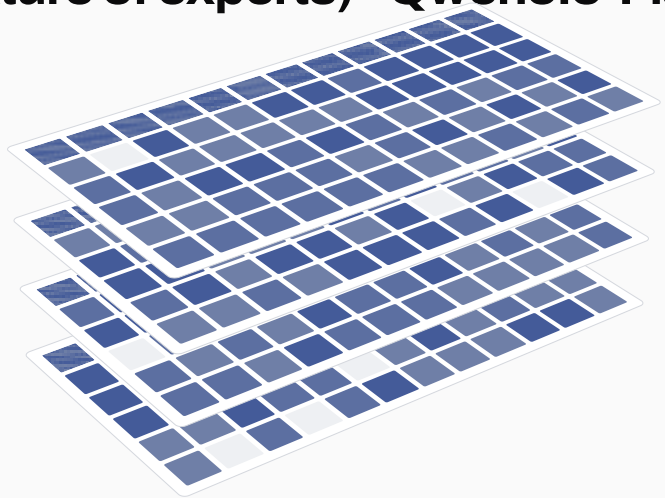
Dense · Qwen3.8-27B



weights attention layer, 1 in 4: writes the KV cache

51 GB decode weights per step · shared across the batch

MoE (mixture of experts) · Qwen3.8-Flash-Next



always read experts, shaded by the request that picked them

231 GB read per step at batch 128: 92% of 250 GB decode weights

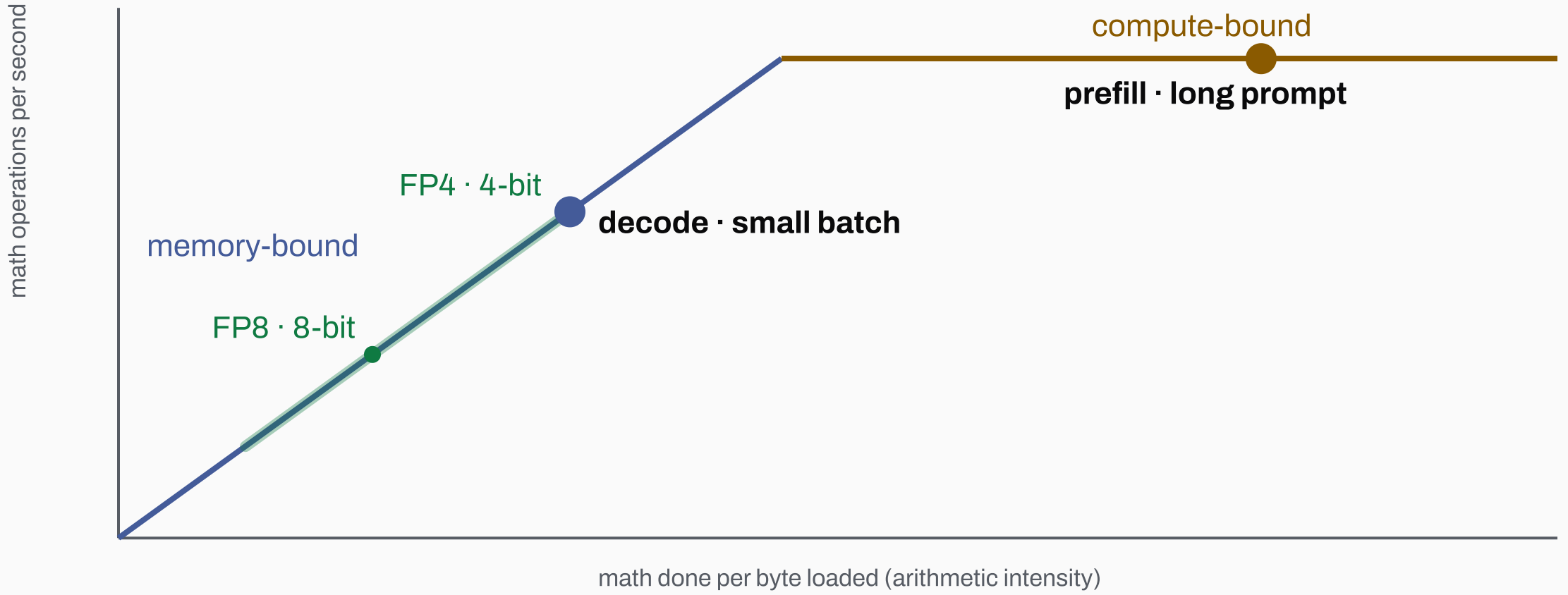
1 Size memory for **all** the weights.

2 Compute the one-user ceiling from the **active** bytes.

3 Under load, plan bandwidth for nearly all the weights.

Four layer planes, schematic. Qwen3.8-Flash-Next config.json: 512 routed experts, top-10, 1 shared. bf16 bytes per decode step: 13.3 GB at batch 1, 231 of 250.2 GB at batch 128, assuming uniform routing (arXiv:2511.02237).

Small-batch decode sits on the slope. Long-prompt prefill sits on the roof.



Schematic. Yuan et al. 2024, "LLM Inference Unveiled", arXiv:2402.16363.

MBU tells you whether tuning can help.

THE MEMORY PIPE · SCHEMATIC



reads mostly back to back
check the profile

MBU: **how full the pipe is.**

`one-user tok/s ÷ one-user ceiling`

High MBU

The bus is likely the limit. Confirm in a profile before you stop tuning overhead. Fewer bytes still raise tok/s.

TRY

```
VLLM_WORKER_MULTIPROC_METHOD=spawn nsys profile \
  --trace-fork-before-exec=true --cuda-graph-trace=node \
  --capture-range=cudaProfilerApi --capture-range-end repeat \
  vllm serve Qwen/Qwen3.8-27B --profiler-config.profiler cuda
```

CHECK

```
vllm bench serve --backend vllm --model Qwen/Qwen3.8-27B --dataset-name random \
  --random-input-len 1024 --random-output-len 512 --num-prompts 2 --max-concurrency 1 --profile
nsys stats report1.nsys-rep
```

Model bandwidth utilization (MBU), Databricks 2023. Short-context, one-user estimate; not a hardware-counter reading. No universal stop-tuning percentage.

Measure your MBU in five minutes.

```
# 1 · Cached, pinned bf16 model; no weight download
python3 - <<'PYCODE'
import json
from pathlib import Path
from huggingface_hub import snapshot_download
root = Path(snapshot_download("Qwen/Qwen3.8-27B",
    revision="1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0",
    local_files_only=True))
weights = 0
for file in root.glob("*.safetensors"):
    with file.open("rb") as f:
        size = int.from_bytes(f.read(8), "little")
        header = json.loads(f.read(size))
    for name, t in header.items():
        if name == "__metadata__" or name.startswith(
                ("model.visual.", "mtp.")): continue
        lo, hi = t["data_offsets"]
        weights += (t["shape"][1] * 2 if name.endswith(
            "embed_tokens.weight") else hi - lo)
print(f"{weights / 1e9:.2f} GB; {273e9 / weights:.2f} tok/s")
PYCODE
```

```
# 2 · Warm server: qwen38 on port 18082
vllm bench serve --backend vllm --model qwen38 \
  --tokenizer Qwen/Qwen3.8-27B --port 18082 \
  --dataset-name random --seed 1234 \
  --random-input-len 128 --random-output-len 128 \
  --max-concurrency 1 --num-prompts 4 \
  --num-warmups 1 --ignore-eos --temperature 0
```

```
# 3 · Use Mean TPOT, not aggregate output tok/s
ceiling = 273 GB/s ÷ 51.25 GB = 5.33 tok/s
measured = 1000 ÷ 222.5 ms = 4.49 tok/s
MBU = 4.49 ÷ 5.33 ≈ 84%
```

Spark · untuned baseline: **222.5 ms** TPOT ≈ **4.49 tok/s** ÷ 5.33 ≈ **84% MBU**.

DGX Spark · Qwen3.8-27B bf16 · vLLM 0.32.0 · 25 Oct 2026. Median: 3 × 16 saved coding prompts, 128 output tokens. Eager, prefix/chunking off. Weight estimate omits vision, MTP, full input embeddings and KV/state traffic. 273 GB/s: NVIDIA. Warm-server diagnostic above; exact runs: handout.

Three things fight for the same GPU memory.



H100 SXM · 79.6 GiB · Qwen3.8-27B bf16 · KV 64 KiB PER TOKEN + 147 MiB PER CONVERSATION

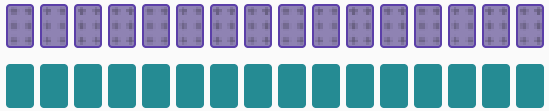
IDEAL PAYLOAD CAPACITY · CONVERSATIONS

2 128k context

Computed, GiB: 79.6 (nvidia-smi) × 0.92 (vLLM default --gpu-memory-utilization) − 51.7 weights (the 55.6 GB checkpoint) − 4 assumed runtime = 17.5 for KV, ÷ (64 KiB × context + 147 MiB state) per conversation. Bar to scale. Measure vLLM's startup capacity.

Do the KV math once instead of guessing.

Every token leaves a **key** and a **value**: two lists of numbers.



key · 256 numbers

value · 256 numbers

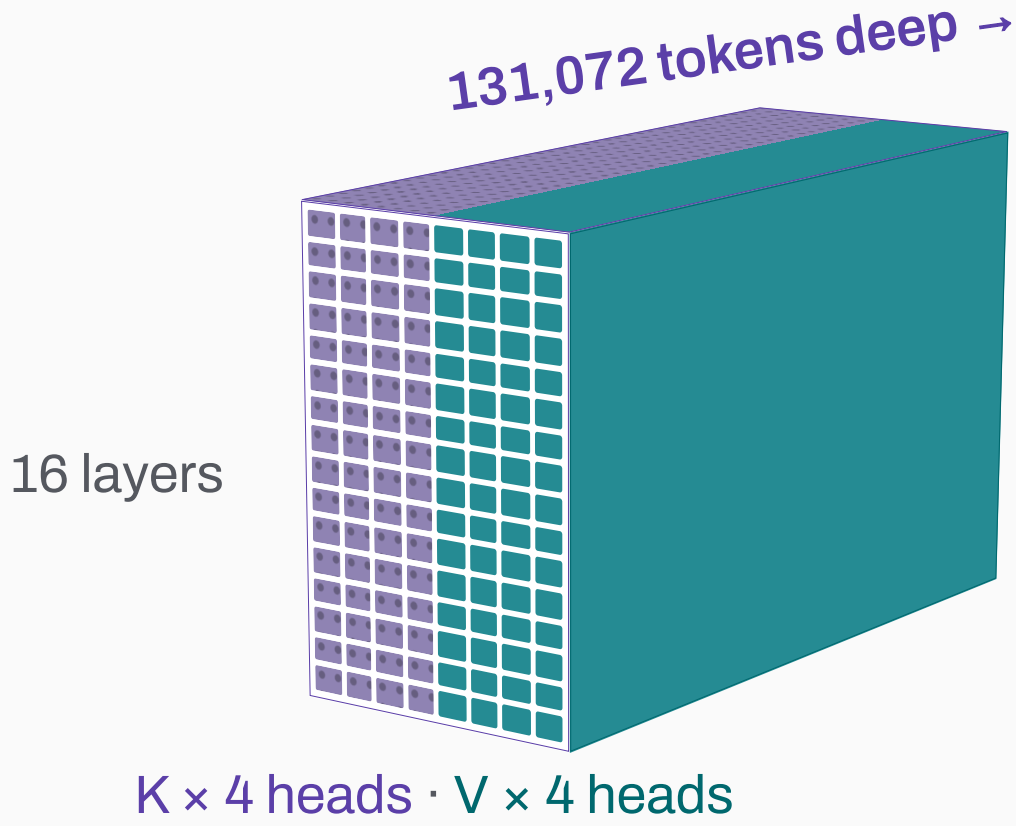
× **16** attention layers × **4** KV heads · each list **256** numbers, **2** bytes apiece

= **64 KiB** per token: one slab

```
2 × 16 × 4 × 256 × 2 B = 65,536 B = 64 KiB
```

× 131,072 tokens = **8 GiB** for one 128k conversation

+ the other 48 layers (Gated DeltaNet): one fixed 147 MiB state.
Payload ≈ 8.1 GiB, before copies and padding.

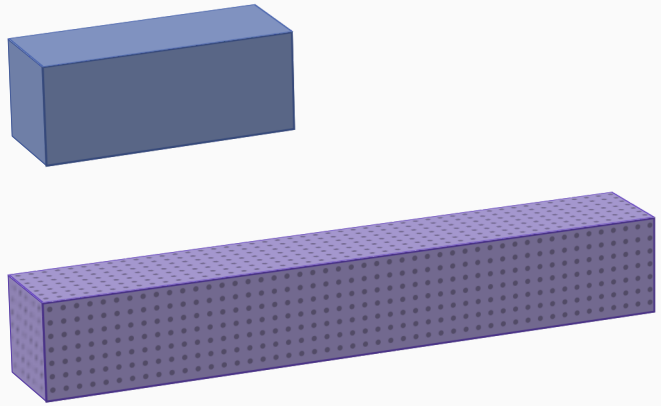


Qwen3.8-27B config.json: 64 layers, 16 full attention (4 KV heads, head_dim 256), 48 Gated DeltaNet with 146.8 MiB fixed state per sequence (fp32 recurrent, bf16 convolution) (vLLM v0.31.0 mamba_utils.py). Strips schematic.

At long context, the KV cache outgrows the weights.

weights **51.7 GiB**
= 55.6 GB

KV · 16 users **130.3 GiB**



2.5× the weights

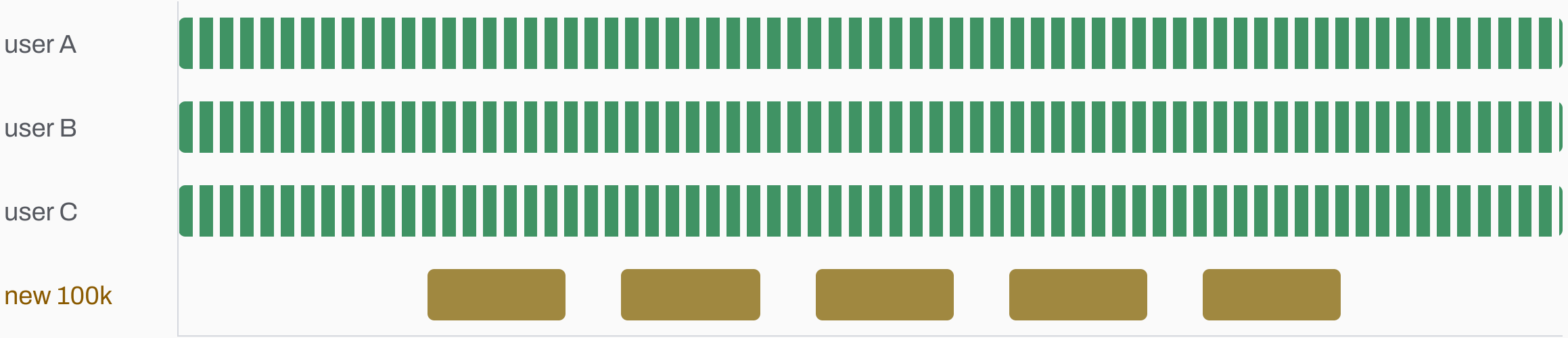
same front face for both: depth is bytes, drawn to scale

4k 32k **128k**

Qwen3.8-27B, 64 KiB per token. The weight footprint stays fixed. The KV doesn't.

Computed: 16 users × (64 KiB × tokens + 147 MiB state) = 6.3 / 34.3 / 130.3 GiB at 4k / 32k / 128k; weights 51.7 GiB (the 55.6 GB checkpoint). Depth to scale; front face schematic.

Unchunked, one 100k prompt stalls everyone else.



Chunked prefill: slices of the prompt, interleaved with everyone's decode.

```
TRY    --max-num-batched-tokens 2048 · 8192 · 16384 (scheduling-step token budget)
CHECK  P95 (95th percentile) vllm:inter_token_latency_seconds vs P95 TTFT (time to first token)
```

Stall timeline schematic. vLLM v0.31.0: chunked prefill on by default; --max-num-batched-tokens defaults to 8192 (H100/H200) and 16384 (B200).

Four levers. Pull the one your traffic needs.

1

Reuse the prefix

skip prefill you already paid for

agents, RAG (retrieval-augmented generation), shared prompts

2

Fewer weight bytes

shrink the per-token read

high MBU, short context

3

Fewer KV bytes

shrink the context read

long context, many users

4

Speculative decoding

more tokens per weight read

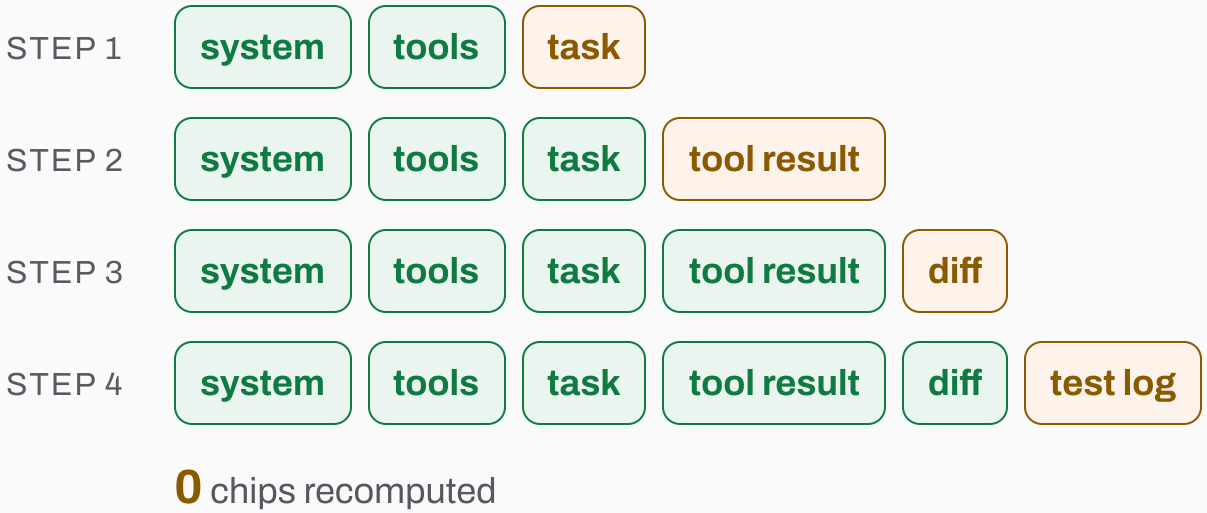
predictable output

skip repeated work

min the bytes

max the tokens per read

An agent resends the same prefix on every step.



CACHE HIT: REUSED PREFIX

$0.022

per 1M cached input tokens

Developers with the same harness and tools can share the prefix.

```
TRY    on by default · compare with --no-enable-prefix-caching
CHECK  sum(rate(vllm:prefix_cache_hits_total[5m])) / sum(rate(vllm:prefix_cache_queries_total[5m]))
```

Turns schematic. Price: DeepSeek V4-Pro, off-peak, checked 25 Oct 2026. Share prefixes within an approved group; use different secret "cache_salt" values across required isolation boundaries (vLLM v0.31.0).

KV offload adds capacity and transfer latency.

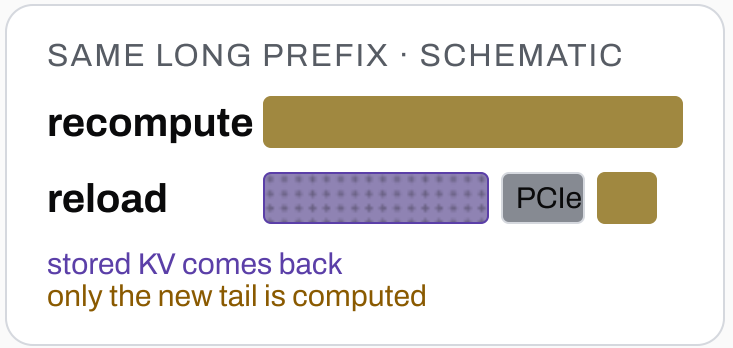
HBM ≈ 3–8 TB/s · tens to hundreds of GB

KV

CPU memory over PCIe 5 x16, the CPU-to-GPU link
≈ 64 GB/s each way

Local flash storage ≈ 7–14 GB/s per drive

Remote storage network-bound · measure reload latency



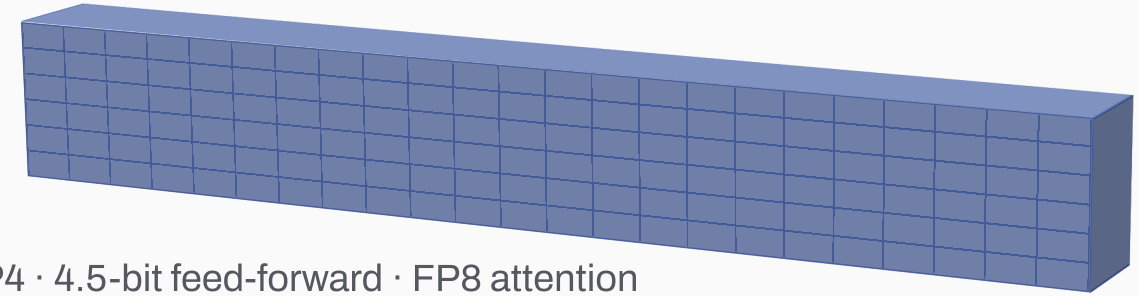
More capacity, with transfer time to pay. Reload only when the transfer beats the recompute: measure it.

```
TRY    LMCache MP · --mamba-cache-mode align · long disk-setup recipe: handout
CHECK  sum(rate(vllm:external_prefix_cache_hits_total[5m])) / sum(rate(vllm:external_prefix_cache_queries_total[5m]))
```

Discrete-GPU tiers, schematic; the Spark shares CPU/GPU memory, so its experiment uses disk without CPU retention. vLLM v0.31.0: CPU offload size is GiB summed across GPUs. Router-visible native offload needs "self_describing_kv_events" and KV event publishing, both off by default. Bandwidths: vendor specs, PCIe 5 x16, Gen4–Gen5 NVMe.

Weight quantization shrinks the weight read. The ceiling rises in proportion.

QWEN3.8-27B CHECKPOINT · SAME WEIGHTS, FEWER BYTES EACH · THICKNESS TO SCALE



NVFP4 · 4.5-bit feed-forward · FP8 attention

21.9 GB

ONE-USER CEILING · B200, 8 TB/s · ONE BOX = ONE TOKEN'S WEIGHTS



365 tok/s

NVFP4 runs W4A4 (4-bit weights, 4-bit activations: the values passed between layers) only on Blackwell (B200). On Hopper, vLLM falls back to W4A16: 4-bit weights, 16-bit activations.

```
BLACKWELL  vllm serve nvidia/Qwen3.8-27B-NVFP4
HOPPER     vllm serve Qwen/Qwen3.8-27B --quantization fp8_per_tensor
CHECK      log "Model loading took X GiB"; a Marlin warning means no 4-bit math
```

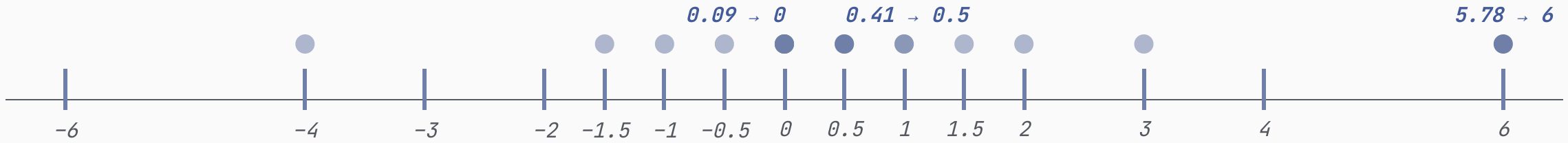
Belt schematic. Sizes: Qwen/Qwen3.8-27B, -FP8, nvidia/Qwen3.8-27B-NVFP4 (NVFP4 feed-forward blocks and lm_head; FP8 attention and DeltaNet). NVFP4: 4-bit values, an 8-bit scale per 16 weights, an FP32 scale per tensor. B200 8 TB/s: NVIDIA spec. vLLM v0.31.0: plain --quantization fp8 no longer quantizes a bf16 checkpoint at load.

NVFP4 stores 4-bit values with two levels of scale.

÷ 0.8125 4.70 ÷ 6 = 0.78, stored as the nearest 8-bit float, once per 16 weights, so the biggest weight lands near 6.

1	0	3	0.5	-1.5	0	6	-1	0	2	-4	0.5	0	1	-0.5	1.5

THE VALUES NVFP4 CAN HOLD



FP32 (32-bit float) tensor scale

BLOCKS OF 16, ONE 8-BIT SCALE EACH

Including scales, each weight uses under a third of bf16 storage.

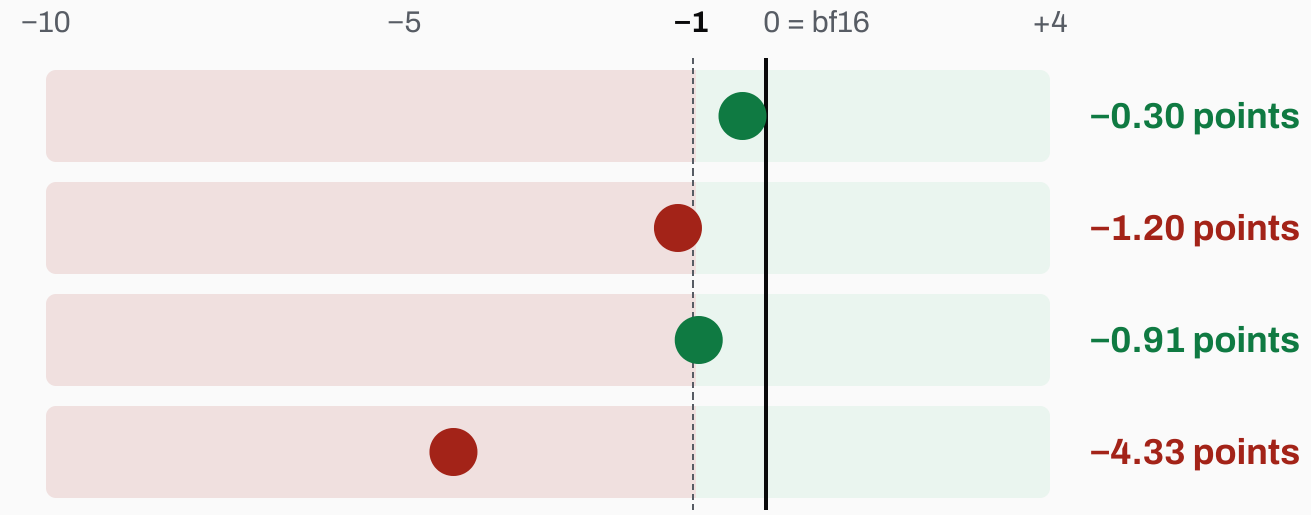
4-bit values plus an 8-bit scale per 16 weights: 4.5 bits per weight.

NVIDIA, "Introducing NVFP4" (Jun 2025) · Four Over Six, arXiv:2512.02010. Example block computed by hand, round to nearest; block scale rounded to FP8 (E4M3).

NVFP4 vs bf16: published score changes.

Example budget: **at most 1 point loss**. Within budget / over budget.

Dots show published means, not certainty about quality.



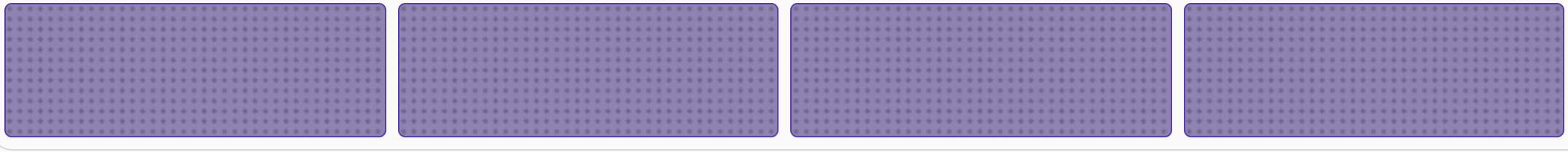
Different quantized layers. Gate on your workload, not on one exam.

```
TRY    lm-eval run --model local-chat-completions --apply_chat_template \
         --model_args model=Qwen/Qwen3.8-27B,base_url=http://localhost:8000/v1/chat/completions \
         --tasks mmlu_pro --num_fewshot 0 --seed 1234 --log_samples --output_path results/baseline
CHECK  repeat both; pair cases/seeds; keep quality loss within your predeclared budget (slide 35)
```

RedHatAI model cards, evaluated on vLLM: Qwen3.8-Flash-Next-NVFP4 (1 seed, expert linears W4A4); gemma-4-31B-it-NVFP4 (3-seed means, block linears W4A4, lm-eval 0.4.11, zero-shot). No paired uncertainty interval is available here.

FP8 KV first. More conversations fit.

IDEAL PAYLOAD BUDGET AFTER WEIGHTS AND RUNTIME: ≈ 17.5 GiB



128k CONVERSATIONS · IDEAL PAYLOAD ESTIMATE **4**

FP8 · roughly half the payload

```
TRY    vllm serve Qwen/Qwen3.8-27B --kv-cache-dtype fp8
CHECK  startup log "GPU KV cache size: N tokens", before and after
```

Qwen3.8-27B at 128k, 17.5 GiB budget (slide 11): 8.1 GiB per conversation in bf16, 4.1 GiB in FP8, each with its unchanged 147 MiB state. Estimates, not measured admission limits.

FP8 KV halves the bytes. It only shows when those bytes were the bottleneck.



vLLM's published burst test · Llama-3.3-70B · 4× H100 · P99 (99th percentile) TTFT

17 s bf16 KV **1.3 s** FP8 KV

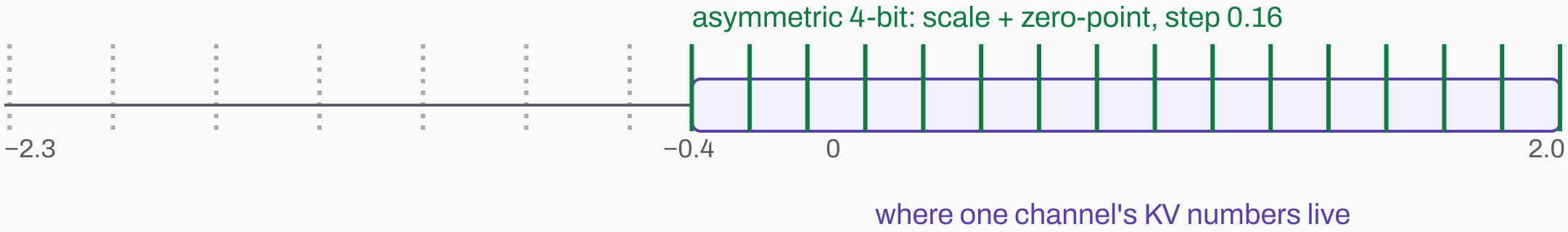
Qwen3-30B-A3B-Instruct-2507 · MoE · 2× H100 · memory to spare

same FP8 KV = bf16 KV

No calibrated KV scales in the checkpoint: vLLM uses a default scale and logs a warning.

vLLM blog, 22 Apr 2026 (crossover, gpt-oss-20b, v0.19.1, concurrency 1; full FP8 crosses at 22,109 on H100, ≈ 13k on B200) and 11 May 2026 (burst TTFT, 300 prompts, 1,024 in / 512 out, v0.20.2). Curve shape schematic. Default scale: vLLM v0.31.0 kv_cache.py.

A KV channel can be off-center. Put the codes where its values are.



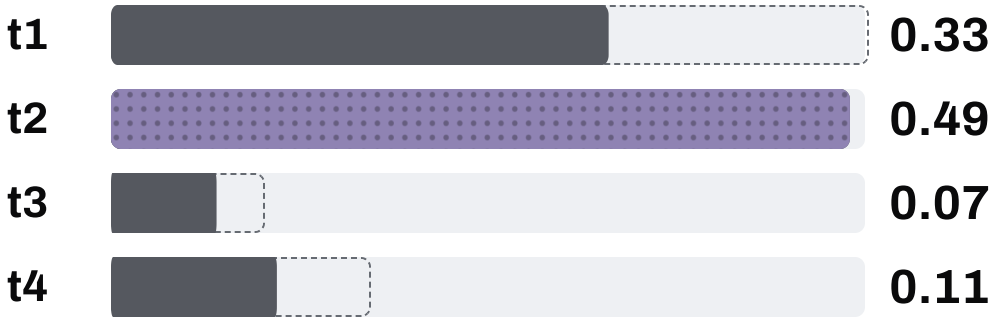
CODES THAT LAND IN RANGE **16 / 16**

At 2 bits there are only 4 codes. Wasting half leaves 2 usable codes.
vLLM's FP8 KV stays symmetric: 8 bits have codes to spare.

Worked example, one channel in [−0.4, 2.0]. Symmetric: scale = 2.0 ÷ 7, codes −8…7. Asymmetric: scale = 2.4 ÷ 15, zero-point shifts the grid. KIVI, arXiv:2402.02750. vLLM v0.31.0 FP8 KV is symmetric only (compressed_tensors.py:1095).

A key error changes who gets attention. A value error changes the mix.

ERROR IN K → CHANGES ATTENTION SHARES



t2's score 1.2 → 2.4: the winner flips

SAME ERROR IN V → AVERAGED BY THE ATTENTION SHARES

ATTENTION SHARES, UNCHANGED



shifts by 0.23 × the error: a change in the mix, not always harmless

6 BITS · KEYS 4 + VALUES 2

10.1

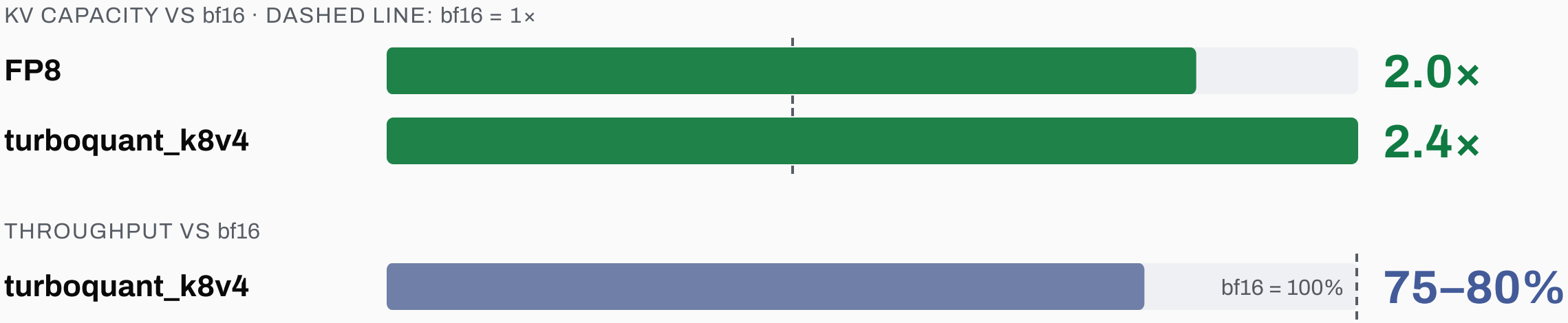
6 BITS · KEYS 2 + VALUES 4

31.5

perplexity, lower is better (8-bit K and V: 9.95). **Give the bits to K, in this measured example.**

Attention shares: illustrative, softmax over scores [2.0, 1.2, 0.4, 0.9]. Perplexity: Wikitext, Transformers KIVI-HQQ, KVTuner arXiv:2502.04420 Table 2 (Llama3-8B-Instruct; KV8 baseline 9.95). QAQ, arXiv:2403.04643, §3.1.

TurboQuant k8v4 uses 8-bit keys and 4-bit values.

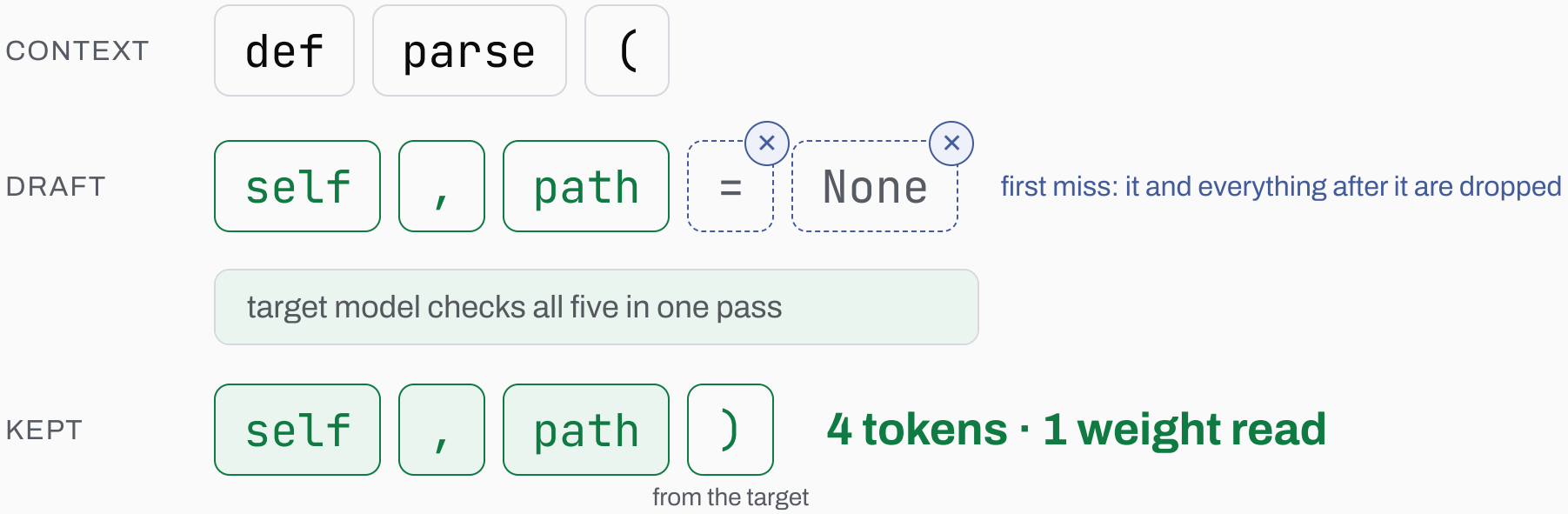


Start with FP8. Reach for k8v4 only when FP8's room isn't enough, and **time it on your GPU** first.

```
TRY    vllm serve Qwen/Qwen3.8-27B --kv-cache-dtype turboquant_k8v4
CHECK  startup log "GPU KV cache size: N tokens", Mean TPOT and P95 TTFT, all vs fp8
```

vLLM blog, 11 May 2026, v0.20.2, offline, 200 prompts: k8v4 throughput 80% of bf16 (Qwen3-30B-A3B, 2×H100) and 75% (Llama-3.3-70B, 4×H100); 2.4× measured capacity; "no significant advantage over FP8". FP8 2.0×: 16 ÷ 8 bits per KV value.

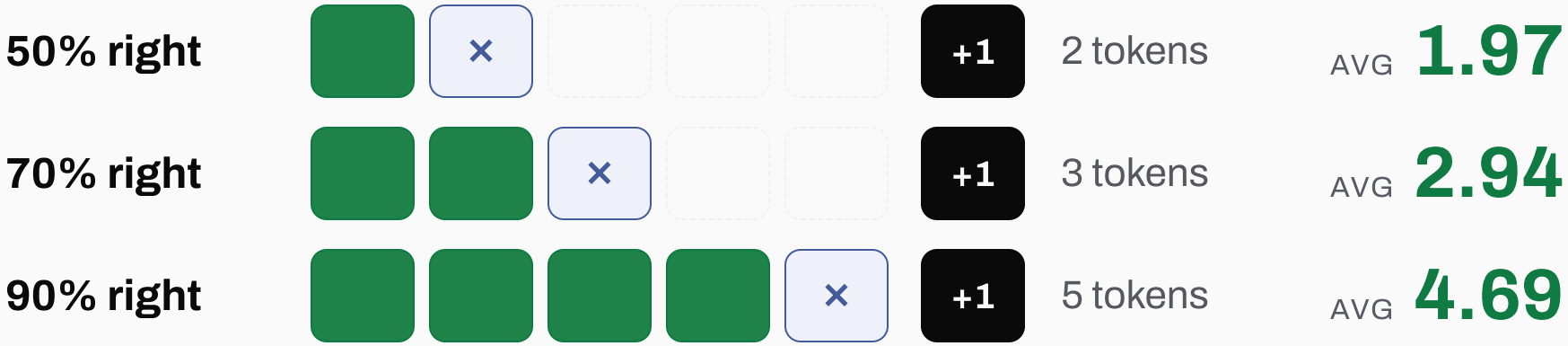
Draft cheap. Verify once. Keep what matches.



Greedy illustration; sampled decoding preserves the target distribution in theory; finite-precision numerics can differ (vLLM v0.31.0 lossless guarantees). Leviathan et al., arXiv:2211.17192: 2×–3× on T5-XXL, batch 1.

Acceptance, not the flag, sets tokens per pass.

Draft 5 tokens. Each is right with some chance. Keep them until the first miss, then the big model adds one of its own.



About 2, 3, 4.7 tokens per pass. vLLM logs this number as `Mean acceptance length`. **Read it on your own traffic.**

More accepted tokens can still be slower. Compare Mean TPOT with speculation off and on; repeat both.

Leviathan et al., arXiv:2211.17192, eq. 1: tokens per pass = $(1 - \alpha^{\gamma+1}) \div (1 - \alpha)$, α = chance a guess is kept, γ = guesses drafted. Here $\gamma = 5$, so 1.97, 2.94, 4.69, if each guess is independent and equally likely to be kept.

Match the drafter to your traffic.

Trained draft layers

EAGLE-3, DSpark or the model's own draft layers (MTP, multi-token prediction). A published Qwen3.8-27B DSpark speculator: **3.76** output tokens per pass, including the target token, on tool calls. **Agent receipt: acceptance, not a measured speedup.**

N-gram lookup

Match the latest tokens to an earlier prompt sequence; propose what followed. No training.

Never

a draft model as slow as the target

N-GRAM SPEEDUP

INSTRUCTIONS	the answer depends on what	1.16×
SUMMARIES	the quarterly revenue rose by	2.42×

MTP · START AT 1

```
vllm serve Qwen/Qwen3.8-27B \
  -sc '{"method":"mtp","num_speculative_tokens":1}'
```

N-GRAM

```
vllm serve Qwen/Qwen3.8-27B \
  -sc '{"method":"ngram","num_speculative_tokens":3,"prompt_lookup_min":2,"prompt_lookup_max":4}'
```

CHECK

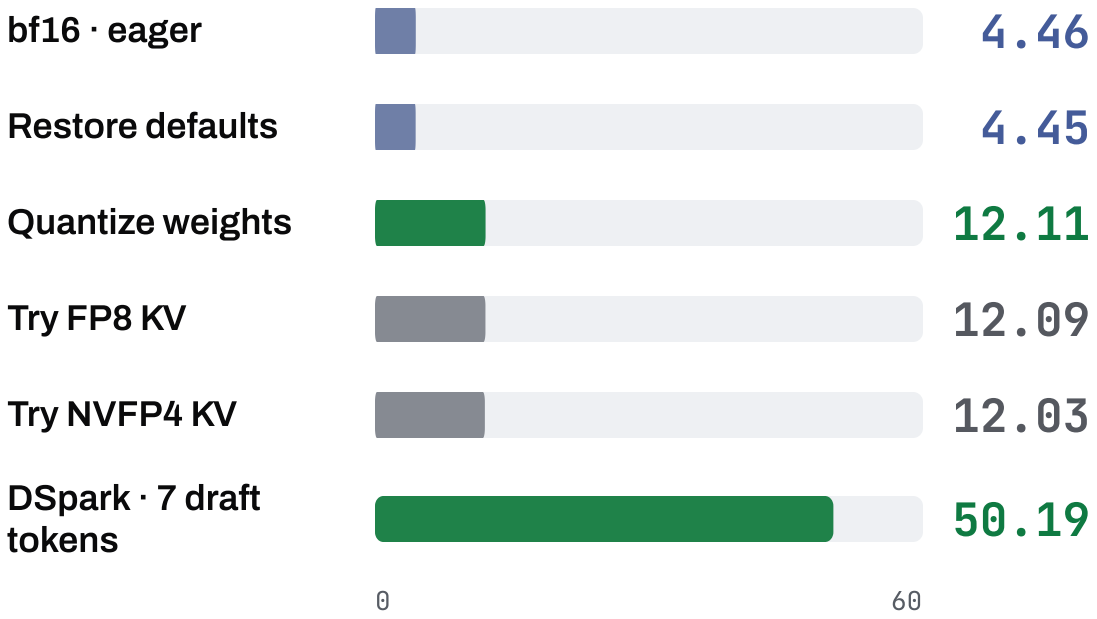
```
log "SpecDecoding metrics: Mean acceptance length" and Mean TPOT
```

Token rows illustrative. n-gram (prompt lookup, Vicuna-13B): Alpaca 1.16×, CNN/DailyMail 2.42× (arXiv:2503.01840, Table 1). RedHatAI/Qwen3.8-27B-speculator.dspark, vLLM 0.29.0, acceptance length including the target token: tool calls 3.76, writing 3.77, math 5.74.

Practical experiment: tune one vLLM deployment

1. Diagnose, then tune

OUTPUT tok/s · 1 CLIENT · 128 OUTPUT TOKENS



Per decode step: **≈9.4 MB of attention KV** vs ≈17.6 GB of weights.
Smaller KV adds room; short decode stays flat.

2. Coding agent spawning subagents

RECORDED REPLAY · MEDIAN 79k IN / 267 OUT

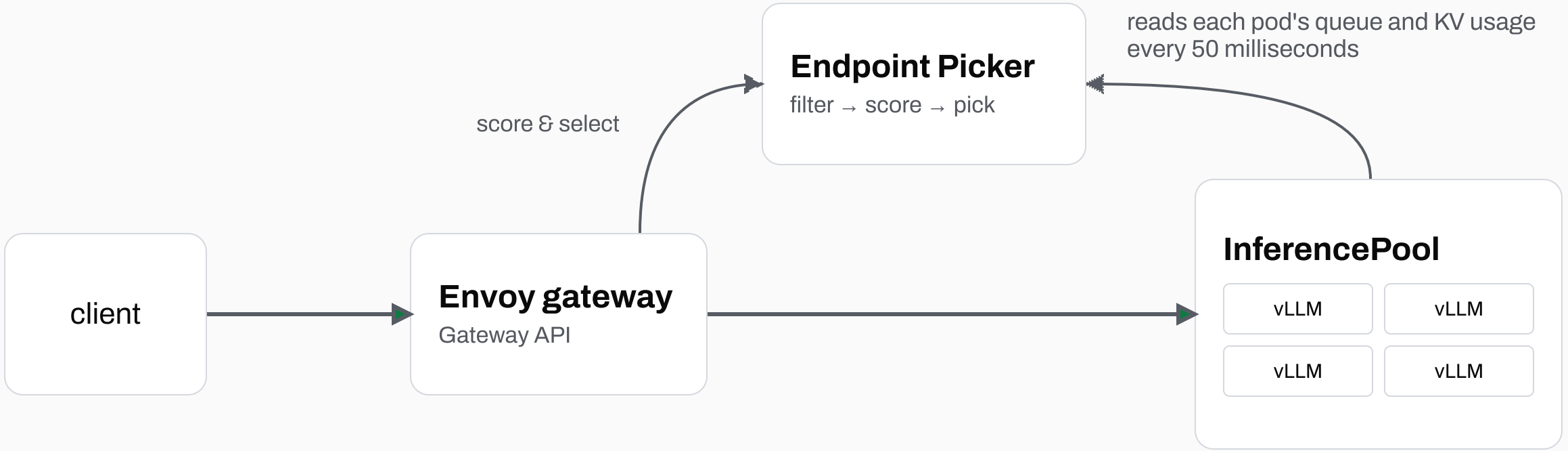
Total output tok/s · below: P95 first-token wait

KV format	8 in flight	16 in flight
Default	**89.45** 8.81 s	**26.16** 239.52 s
FP8	**79.59** 18.75 s	**77.78** 52.02 s
NVFP4	**53.17** 42.54 s	**53.04** 104.55 s

4 in flight, default KV: **74.51 tok/s · 5.31 s P95**.
Native MTP · 7 draft tokens · priming excluded.

Qwen3.8-27B · GB10 · vLLM 0.32.0 · 25–26 Oct 2026. Medians of 3 × 16 requests. Left: drafter uses default KV. Right: recorded replay; priming excluded. Different traffic: do not multiply gains. Long-context quality unchecked. Runs and ranges: handout.

A prefix-aware router knows which prefix each pod has cached.



Precise routing: KV block events → index of block locations; match the request's prefix.

Schematic; example pinned to llm-d v0.10 (29 Sep 2026), CNCF Sandbox; Gateway API Inference Extension v1. Poll: --refresh-metrics-interval, default 50 ms; queue and KV usage are load, not prefix identity. Same pattern: NVIDIA Dynamo (--router-mode kv), vLLM Production Stack.

Random routing repeats each shared prefix's prefill across pods.

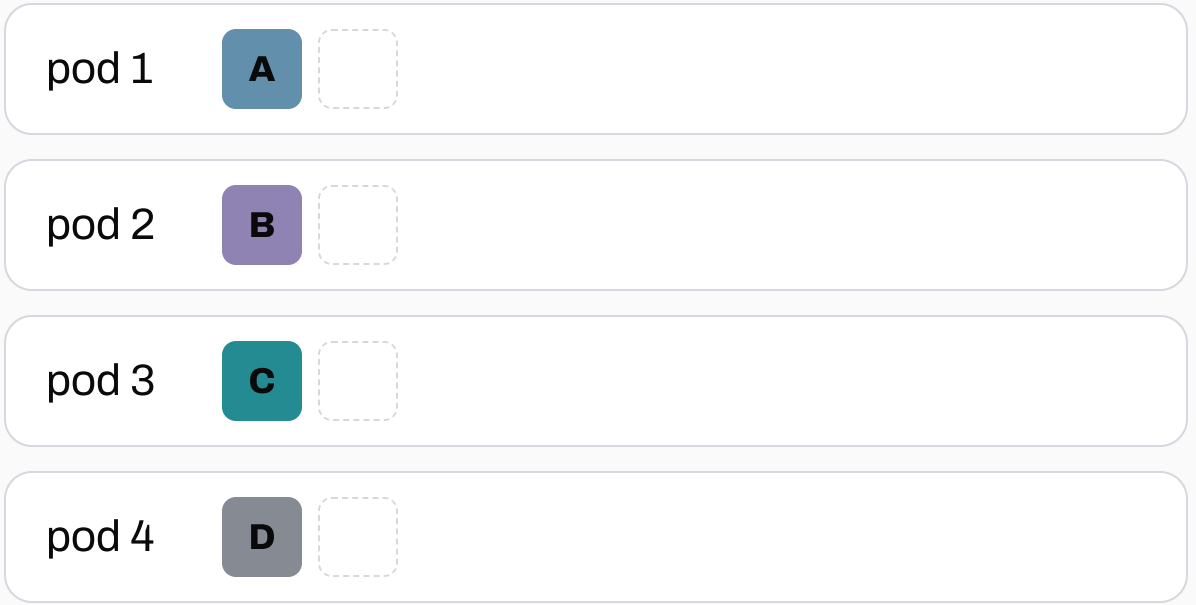
PREFIX-AWARE ROUTING: EACH PREFIX STICKS TO ITS WARM POD

llm-d BENCHMARK · TTFT P90 (90th PERCENTILE)

92.6 s → 0.54 s

OUTPUT tok/s

4,429 → 8,730

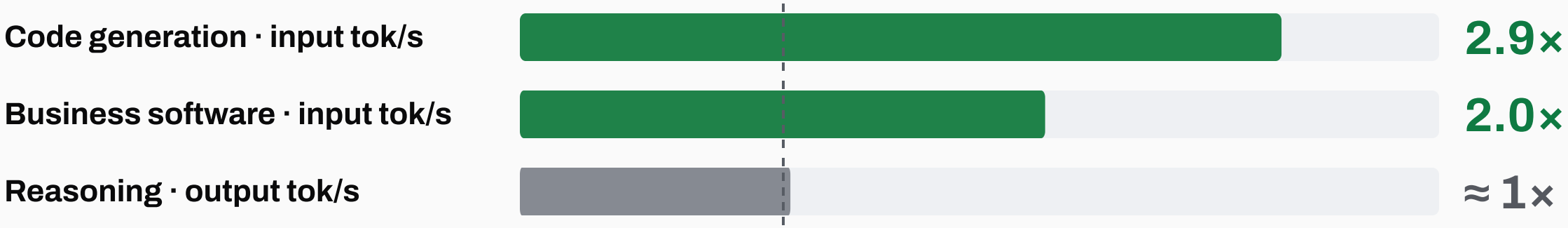



```
TRY    precise-prefix-cache-routing · helm chart llm-d-router-standalone v0.11.0
CHECK  prefix token hit rate per pod, before and after
```

llm-d blog, 24 Sep 2025: 8 vLLM pods × 2 H100, 32B Qwen, 150 tenants × 6,000-token shared prefix. Random vs precise prefix-aware routing.

Three workloads. Matched replicas in each test.

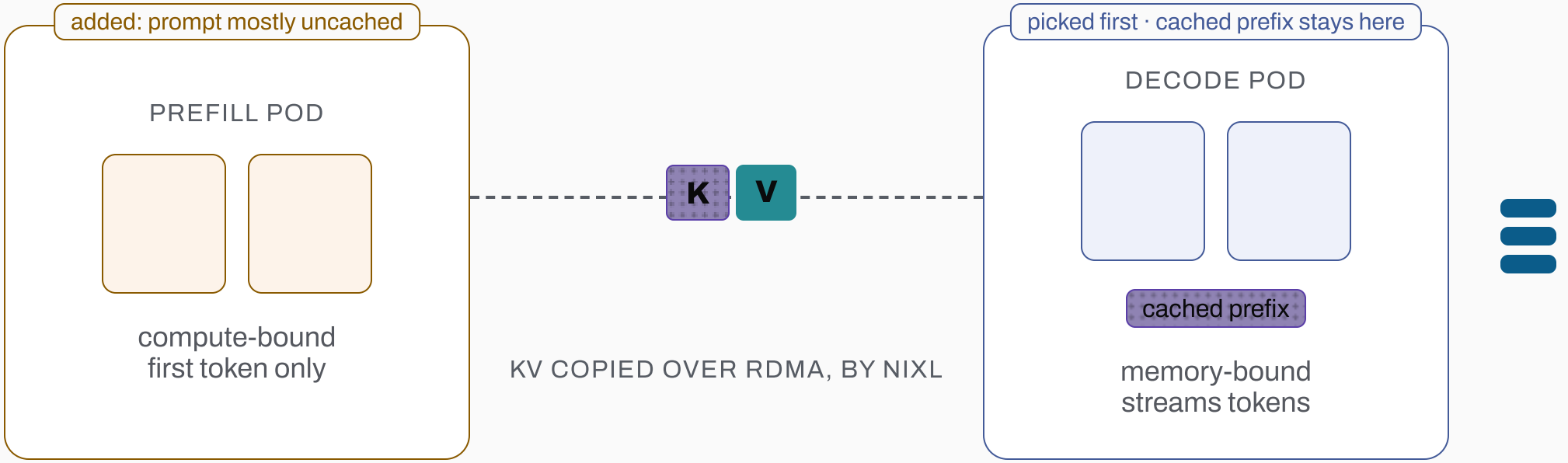
llm-d's default router vs Kubernetes round-robin, same replica count per test. Dashed line: round-robin, 1×.



Reasoning averages about 8,000 output tokens per request. No router shrinks that read.

llm-d blog, 17 Aug 2026. Throughput at the knee. Qwen3-32B, TP=2 (tensor parallel: each layer split across two H100s). Matched pods within each comparison: ten for code and reasoning, eight for business software. Code-gen: about 70,000-token prompts, 46k vs 16k input tok/s; business software 90k vs 45k. Reasoning 7.0k vs 6.8k output tok/s, about 8,000 output tokens on average.

Long prompts get their own prefill pods.



RDMA: direct GPU-memory copies over the network. NIXL: the library that moves the KV between GPUs.

The picker chooses the decode pod first, and adds a prefill pod **only when much of the prompt is uncached**.

Schematic. llm-d prefill/decode (P/D) docs: for 10k input tokens / 1k output tokens, not 200 input / 200 output. Standard-network fallback is for testing only. Splitwise arXiv:2311.18677 · DistServe arXiv:2401.09670.

Alert on latency, cache pressure and queues.

< 500 ms

example · time to first token (TTFT) P95,
interactive; set it and per-request TPOT
P95 from your SLO (service-level objective)

Cache hits

live · prefix token hit rate, by pod and workload

MBU

before and after a change
one-user MBU per model and GPU

$ / 1M

before and after a change
GPU $/hour ÷ output tokens/hour × 1M

busy GPU 100%



queue: 2 waiting

overloaded GPU 100%



queue keeps growing

Scale on active requests and queues, not GPU utilization. **Alert on** KV usage, queue depth, P95 TTFT and per-request TPOT.

500 ms is an illustrative latency target. Set targets from your SLO. Hit rate counts tokens; there is no universal target. Scrape /metrics: vllm:kv_cache_usage_perc and vllm:num_requests_waiting. Busy vs overloaded schematic; llm-d router queues need flow control.

The eval recipe: one change, same traffic, judged on a repeat.

1 Freeze

Save prompts, checks and setup. Set the quality-loss budget. Change **one** thing.

2 Repeat both

Repeat baseline and candidate. Match traffic, warm-up and cache state, including persistent tiers.

3 Compare paired cases

Pair the same cases and seeds. Score the exam and your task separately, with uncertainty intervals.

4 Verdict

- **keep** repeatable gain **and** quality interval within budget
- **revert** no repeatable gain
- **revert** quality loss clearly exceeds budget
- **hold** uncertain: keep baseline, gather evidence

Measure first. Then pull the levers your traffic needs.

For decode. Estimate the ceiling, then one-user MBU.	Short context, speculation off. Low MBU: profile overhead. Batch within the latency target; a `guidellm` sweep finds the knee.
Repeated prefixes	Prefix cache (on by default). Then route to it.
Weights dominate	FP8 where the GPU has FP8 math (an FP8 checkpoint, or `--quantization fp8_per_tensor`); NVFP4 where it has FP4 math (Blackwell).
Context dominates	`--kv-cache-dtype fp8`. Then fewer bits, carefully.
Predictable output	Speculative decode: `mtp`, `eagle3`, `dspark` or `ngram`.
Long prompts, strict TTFT	`--max-num-batched-tokens` first. A prefill/decode split needs RDMA.
Keep only repeatable gains after the quality gate passes.	High MBU, batching plateaued, nothing above applies: stop. No sampler, CUDA graph or sequence-limit tweak shrinks the read.

vLLM v0.31.0 commands and pinned tool versions: field guide.

Your GPU isn't slow. It's starving.

Skip repeated work.
Min the bytes.
Max the tokens per read.

tok/s × bytes = bandwidth

Marcel Wysocki · marcel.w.wysocki@gmail.com



Handout

Run the field experiment yourself

Message “guide” on LinkedIn

Session feedback

The Memory-Bandwidth Wall: A Field Guide to Min-Maxing LLM Serving with vLLM

For this session and the organizers

sfeedback.com/Wj5q8l

Two prices. Same million tokens.

30×

CACHE MISS

$0.66

per 1M input tokens

whole prompt computed

the GPU **computes**

CACHE HIT

$0.022

per 1M input tokens



the GPU **remembers**

DeepSeek V4-Pro API, off-peak · checked 25 Oct 2026 · api-docs.deepseek.com/quick_start/pricing · work strips schematic.

Throughput decides whether the box pays rent.



OUTPUT tok/s ACROSS THE BATCH

1,916

Revenue = output tok/s × 3600 × ($3.48 + 2 × $1.74) per 1M: 2 input tokens per output token, every input token billed at the uncached rate, DeepSeek's former list price. Rental example: Runpod Community Cloud $5.98/GPU-hour; CoreWeave $8.60 (25 Oct 2026).

These flags can't move a bandwidth-bound decode.

~~Sampler settings~~

Sampler settings don't shrink the weight read.

~~CUDA graph tweaks at high MBU~~

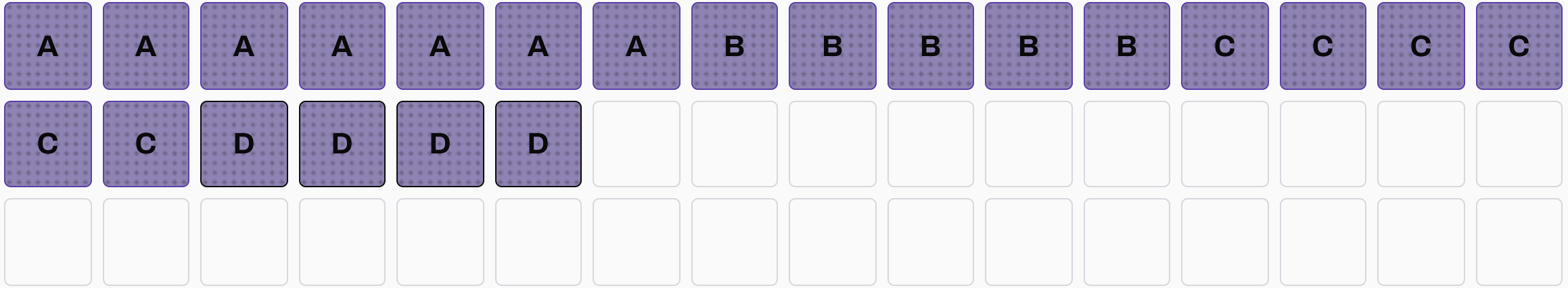
Launch overhead was the low-MBU problem.

~~More `--max-num-seqs` after throughput plateaus~~

Extra sequences still need KV room.

Reserve KV up front and most of it sits empty.

BLOCKS OF 16 TOKENS, ALLOCATED AS TOKENS ARRIVE



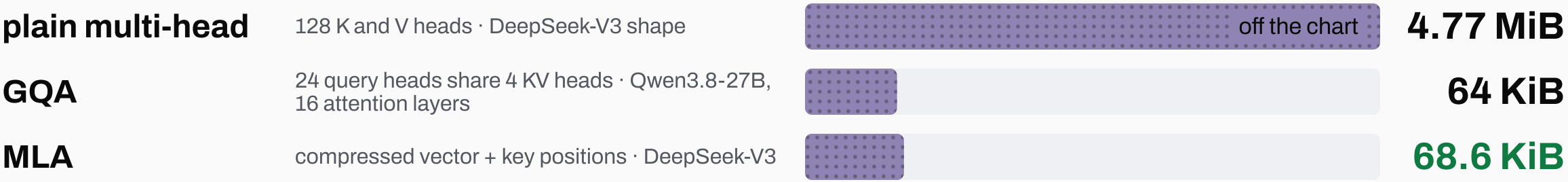
The paper's Orca baselines used 20–38% of KV memory for tokens. vLLM: near-zero waste, 2–4× throughput.

Kwon et al., 2023, arXiv:2309.06180: 20–38% utilization (Fig. 2, Orca variants), 2–4× throughput, OPT and LLaMA on A100s. Layout schematic, 16-token blocks.

DeepSeek V3 caches 71× less than plain attention would.

Same idea, fewer bytes: **share** K and V across query heads, or **compress** them into one small vector and rebuild them when needed.

KV BYTES PER TOKEN · AXIS ZOOMED ABOUT 13×: 0 TO 400 KIB



DEEPSEEK'S FP8 CACHE, ONE LAYER: 656 B (512 FP8 + 16 SCALES + 128 ROPE) INSTEAD OF 1,152 B



DeepSeek-V3 config: kv_lora_rank 512, qk_rope_head_dim 64, 61 layers; plain multi-head = 128 × (192 + 128) × 2 B × 61. Qwen3.8-27B config: 16 × 4 KV heads × 256 × 2 × 2 B = 64 KiB; 146.8 MiB fixed DeltaNet state extra. vLLM v0.31.0 fp8_ds_mla layout (V3.2).

DeepSeek V4 compresses the sequence, not just the vector.

tokens

CSA · 4 → 1



a learned summary of every 4 tokens

indexer

V4-Pro: a small scorer picks **1,024 entries** for main attention. Flash uses 512. The indexer still reads.

HCA · 128 → 1

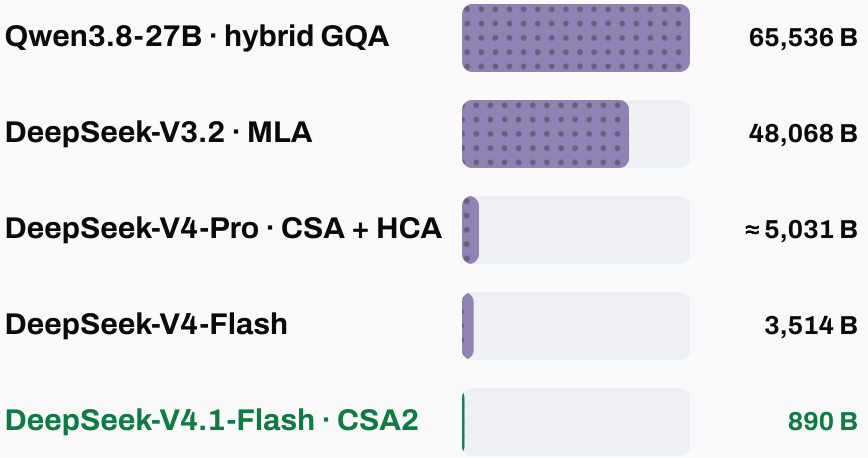


every other layer: a coarse map of everything, read in full

recent

the **last 128 tokens stay uncompressed**: a query can't see its own unfinished block, so recent tokens stay whole

GLOBAL KV BYTES PER TOKEN. EXCLUDES FIXED WINDOW AND STATE. MIXED PRECISION.

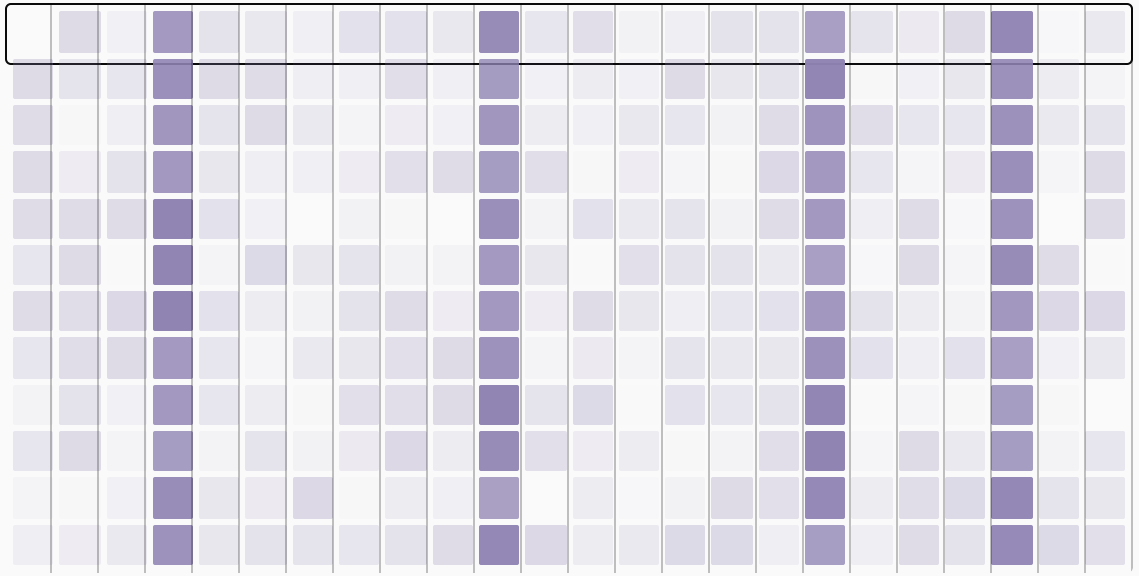


CSA2: layers share entries and picks, no HCA, FP4 cache. 74× less than Qwen3.8-27B's growing KV.

Token rows schematic. DeepSeek-V4 report, arXiv:2606.19348 · V4.1-Flash report, Fig. 1b. V4-Pro computed from config (MXFP4 indexer; FP8: about 5,511 B). Qwen3.8-27B bf16: 16 full-attention layers; fixed DeltaNet state excluded.

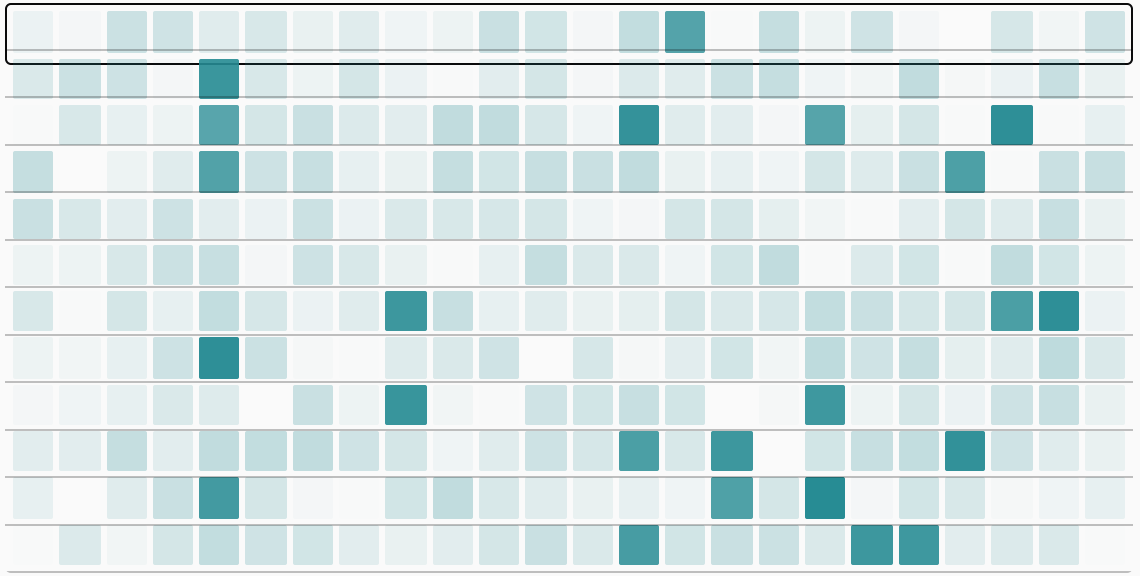
K and V are not the same tensor.

KEYS · OUTLIERS BY CHANNEL



quantize **per channel** ↓

VALUES · OUTLIERS BY TOKEN



quantize **per token** →

Keep the first token wide. At 2-bit nonuniform, without sparse outliers: perplexity 23.3 without it, 9.6 with it. 16-bit: 4.6.

KIVI, arXiv:2402.02750 · KVQuant, arXiv:2401.18079, Table 13 (Llama-2-13B, Wikitext-2): with 1% sparse outliers, keeping the first token wide changes perplexity from about 4.85 to 4.78. Heatmaps are schematic.

vLLM serves the request. llm-d routes across pods.

API + scheduler
continuous batching, chunked prefill

PROBLEM 3

Prefix cache
reuse KV for repeated prefixes

LEVER 1

PagedAttention
KV in blocks, near-zero waste

ENGINE CORE

KV quantization
FP8, TurboQuant

LEVER 3

Kernels
FlashAttention, FP8, NVFP4

LEVER 2

Spec decode
draft, verify, keep

LEVER 4

TP / PP / EP
tensor, pipeline, expert parallel: one model, many GPUs

SCALE

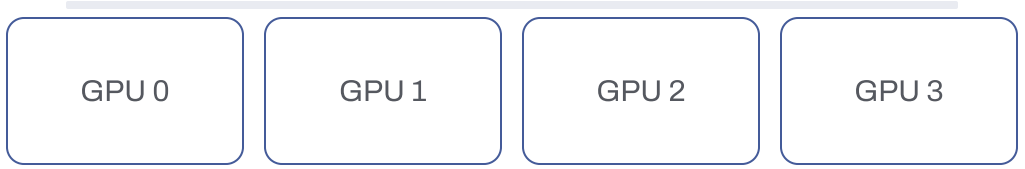
llm-d
routing and prefill/decode (P/D) across pods

SCALE

Schematic. PagedAttention waste: Kwon et al., arXiv:2309.06180.

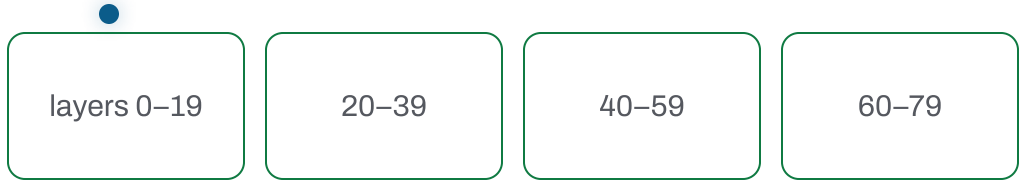
TP splits a layer. PP splits the model.

Tensor parallel (TP)



Every layer, all GPUs, in lockstep. All-reduce, an exchange and sum of partial results, on every layer.
Wants NVLink. Inside the node.

Pipeline parallel (PP)



Layer ranges in turn. Activations, the intermediate values, hop once per stage.
Less communication. Measure across nodes.

Schematic. All-reduce exchanges and sums each GPU's partial results.

Start with the baseline. Add recipes for your workload.

Optimized baseline

Envoy + Endpoint Picker + vLLM pods

start here · any fleet

Tiered prefix cache

HBM → CPU memory → storage

big shared prefixes

P/D disaggregation

prefill pods → KV over NIXL (GPU-to-GPU transfer) → decode pods

long prompts, strict TTFT, RDMA

Wide expert parallel

16 prefill + 16 decode B200: ≈ 3.1k output tok/s per decode GPU

DeepSeek-class MoE · batch processing, relaxed latency

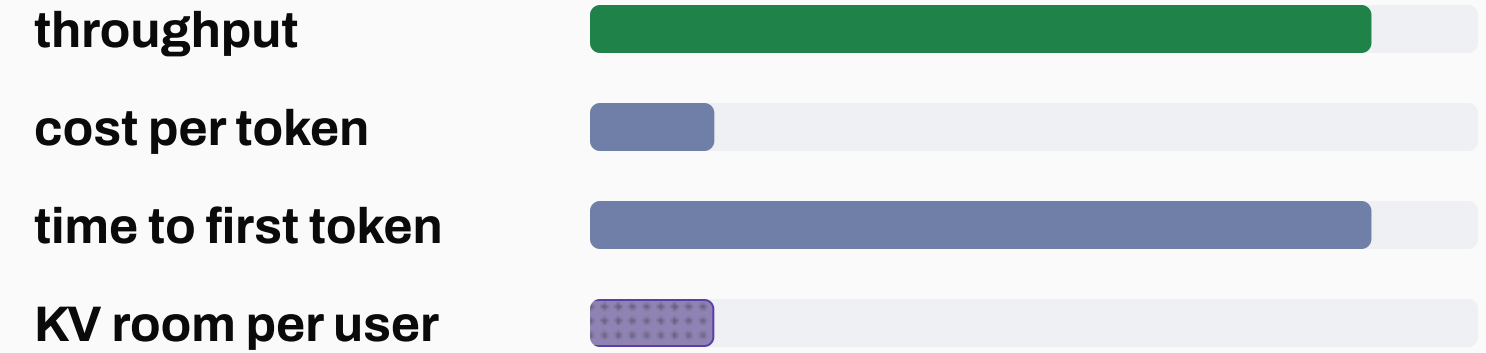
Scale on active requests and queues, not GPU utilization: busy and overloaded look the same.

llm-d "well-lit path" guides. Wide-EP: llm-d v0.5 blog, random 1k/1k. Autoscaling: KEDA on Endpoint Picker metrics.

Bigger batches trade latency and KV room for throughput.

BATCH SIZE

1 many

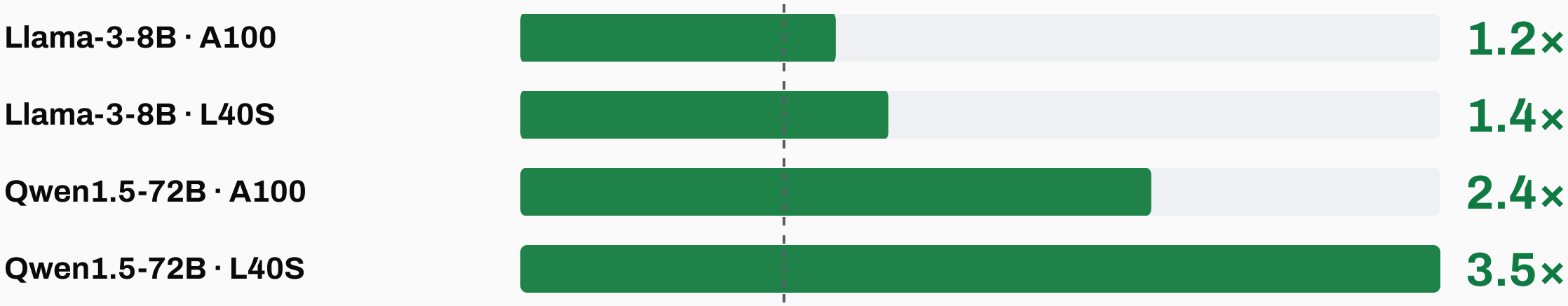


Four numbers move together. **Change one variable at a time.**

Schematic. Directions, not measurements.

QServe improved the whole serving system.

Existing 4-bit integer methods paid 20–90% runtime overhead for dequantization in large-batch serving. QServe's system gains. Dashed line: TensorRT-LLM baseline, 1×.



Lin et al., arXiv:2405.04532: maximum throughput, 1,024 in / 512 out, same memory per device, against the best TensorRT-LLM v0.9.0 configuration (FP16, W8A8 or W4A16). Whole-system comparison.

Report cache-hit rate alongside input throughput.

DEEPSEEK V3/R1 · INPUT THROUGHPUT PER EIGHT-H800 NODE



73.7k tok/s

A hit still costs find the block · read it · move it to the right GPU · compute the tail

Ask every vendor for the hit rate behind that number.

DeepSeek Open Infra Index, Day 6 (Feb 2025), 24-hour averages per eight-H800 node: 73.7k input tok/s in prefill or 14.8k output tok/s in decode; 56.3% of input tokens hit the on-disk KV cache.